FoodRescue360

SMART FOOD REDISTRIBUTION SYSTEM

PROJECT REPORT

Submitted by

Deepika Gupta

Roll Number: 31

Lovely Professional University

Project Guide

Dr. Ashu

Academic Year 2026–2027

Declaration

I declare that the project titled “FoodRescue360 — Smart Food Redistribution System” has been prepared for academic purposes. This report describes the application, its MongoDB/PyMongo data layer, and its machine-learning prototype. The dataset scope and model limitations are stated clearly.

Student: Deepika Gupta | Roll Number: 31

Signature: __Deepika Gupta _____ Date: ___25-10-2026___

Acknowledgement

I would like to thank my project guide, Ansh Pratap Singh, and Lovely Professional University for the opportunity to work on this project. I also acknowledge the documentation and open-source communities behind Python, FastAPI, React, MongoDB, PyMongo, pandas, and scikit-learn.

Abstract

FoodRescue360 is a web-based prototype designed to make surplus-food redistribution easier to coordinate. Donors can create food listings with quantity, location, and expiry details, while NGOs can register, find nearby organisations, and claim available donations. The application tracks listing status and displays summary information on a dashboard. React and Vite provide the interface, FastAPI handles API requests, and MongoDB Atlas stores records through PyMongo. A Random Forest Regressor estimates general food-delivery duration from a 1,000-row CSV dataset. The recorded held-out test results are R^2 = 0.7961 and MAE = 6.67 minutes. Because the dataset describes general food delivery rather than food-rescue pickups, the estimate is a learning prototype, not a real-world guarantee.

1. Introduction

Food is sometimes left unused by households, restaurants, events, and other food providers, while community organisations may be looking for donations to support people in need. The challenge is not always a lack of willingness to donate; it can also be the difficulty of sharing current information and coordinating collection at the right time.

When donation details are shared through informal messages or separate spreadsheets, a listing can become outdated, two organisations may ask for the same food, and expiry information can be missed. A central application can make the process more organised by keeping donation records, NGO details, and claim status in one place.

FoodRescue360 was developed as a practical student project to explore this idea. The web interface supports food listing and NGO workflows. MongoDB stores the records in a flexible document format, while PyMongo allows the Python backend to work with those records. A separate machine-learning component estimates general delivery time from historical delivery features.

2. Problem Statement, Aim, and Objective

Problem statement

The project addresses the need for a structured way to record surplus-food donations, make those listings discoverable to NGOs, and keep track of whether food is available or has already been claimed. Manual coordination can make it harder to keep details current, identify nearby organisations, and prioritise food that is close to expiry.

Aim

To build a web application that supports surplus-food donation coordination using MongoDB and PyMongo, and demonstrates a machine-learning regression model for estimating general delivery duration.

Objectives

- Create food listings containing food name, quantity, unit, donor, location, and expiry time.
- Store food listings and NGO profiles in MongoDB Atlas using PyMongo.
- Support NGO registration and nearby-organisation search using coordinates.
- Provide dashboard counts that summarise activity.
- Train a Random Forest Regressor using delivery-related input features.
- Evaluate regression performance with R^2 and MAE and explain the suitability of classification metrics.

3. MongoDB Atlas and PyMongo Implementation

MongoDB Atlas is used to store the data of FoodRescue360 online. I created a cluster named **FoodRescue** in MongoDB Atlas for this project. I also added my current IP address in Network Access to allow my computer to connect to the cluster.

For connecting the project to MongoDB, I selected **Python** as the programming language and **PyMongo** as the client library. PyMongo helps the Python backend add, view, and update data in the database.

The project uses three main collections: food_listings to store food donation details, ngos to store NGO information, and delivery_training_data to store delivery dataset records.

The MongoDB connection string is saved in a local .env file. This allows the backend to connect to the database without writing the connection details directly in the code.

These steps help FoodRescue360 store and manage its data in MongoDB Atlas.

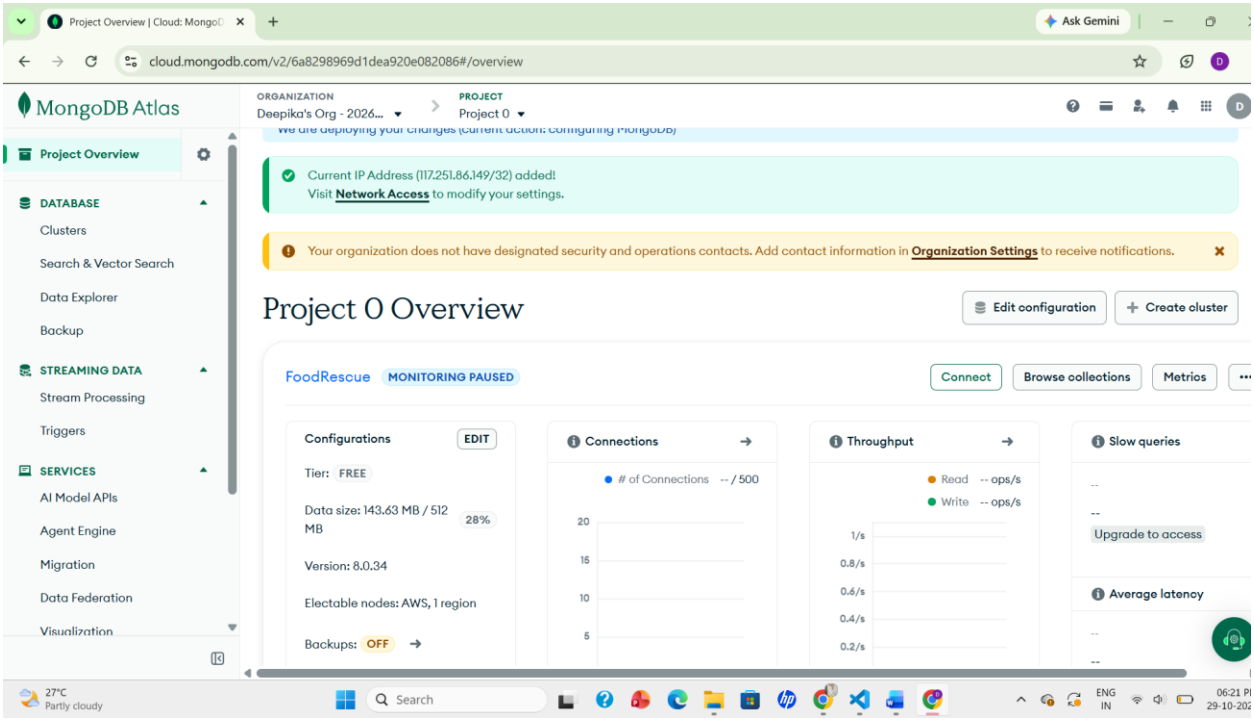


Figure 1. MongoDB Atlas Project Overview showing the FoodRescue cluster and IP access configuration.

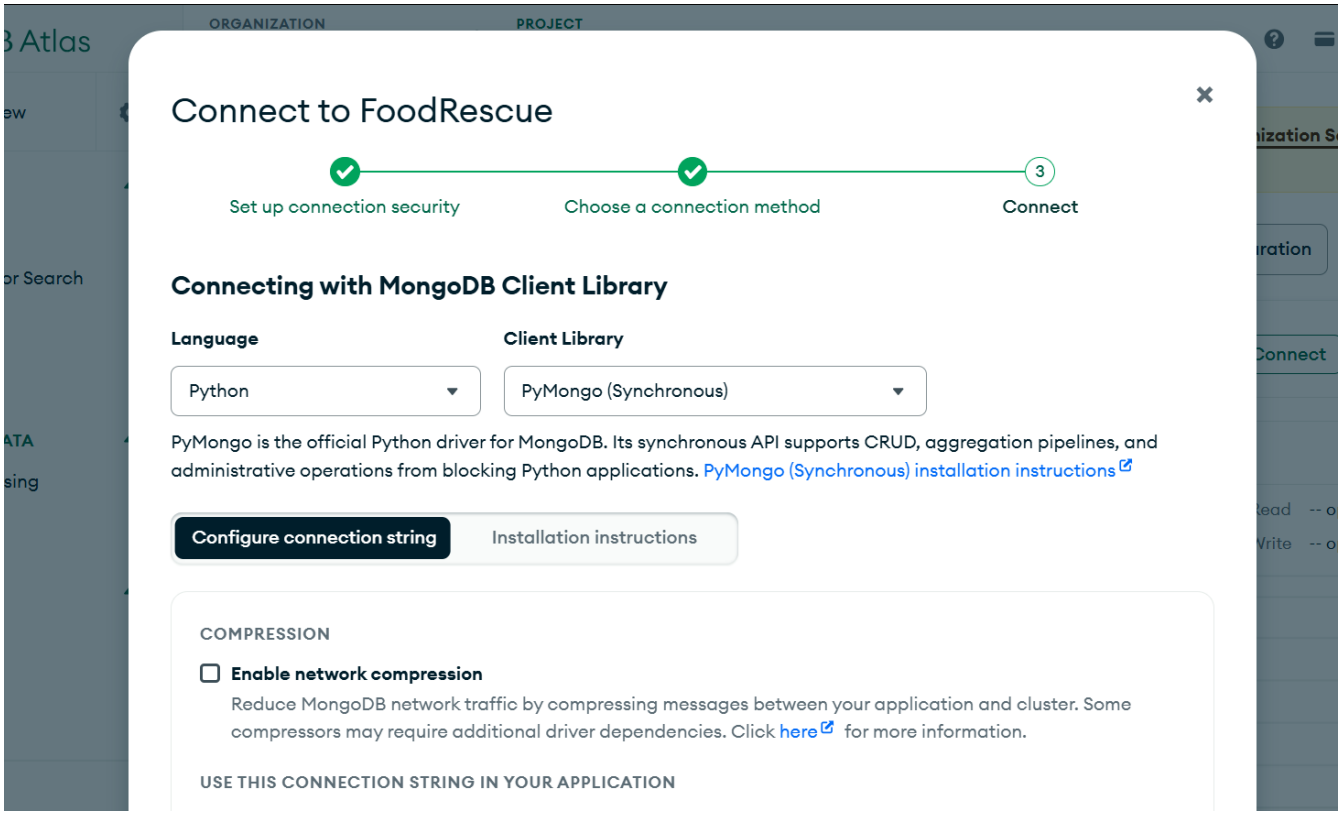


Figure 2. Connect to FoodRescue — Python language aur PyMongo client library selection.

Collections used

Collection	Typical fields / content	Purpose
food_listings	food_name, quantity, unit, donor_name, location, expires_at, status, claim details	Donation listing and status workflow
ngos	name, contact_person, phone, address, location, verified	NGO profiles and location matching
delivery_training_data	Imported CSV row fields, including Order_ID where available	Store imported delivery data in MongoDB

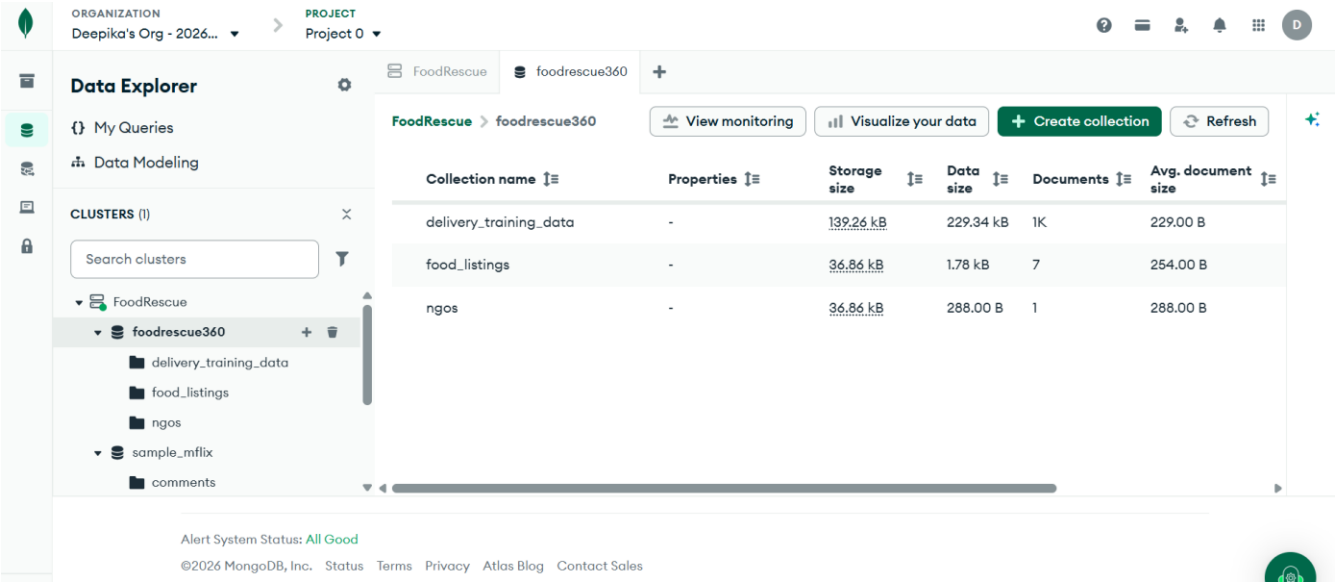


Figure 3. MongoDB Atlas Data Explorer showing the FoodRescue360 database and its collections: delivery_training_data, food_listings, and ngos.

4. Dataset Description and Preprocessing

The machine-learning component reads Food_Delivery_Times.csv from the project's data folder. The file contains 1,000 rows and nine columns, including Order_ID, seven model features, and the target Delivery_Time_min. The target is a continuous numeric value representing delivery time in minutes.

Dataset Link: https://www.kaggle.com/datasets/denkuznetz/food-delivery-time-prediction?utm_source=chatgpt.com

Column	Type	Use
Distance_km	Numeric	Distance of the delivery
Preparation_Time_min	Numeric	Time spent preparing the order
Courier_Experience_yrs	Numeric	Courier experience in years
Weather	Categorical	Weather condition
Traffic_Level	Categorical	Traffic category
Time_of_Day	Categorical	Time period
Vehicle_Type	Categorical	Vehicle used for delivery
Delivery_Time_min	Numeric target	Observed delivery duration
Order_ID	Identifier	Not used as a model feature

	A	B	C	D	E	F	G	H	I
1	Order_ID	Distance_km	Weather	Traffic_Level	Time_of_Day	Vehicle_Type	Preparation_Tim	Courier_Experience_y	Delivery_Time_min
2	522	7.93	Windy	Low	Afternoon	Scooter	12	1	43
3	738	16.42	Clear	Medium	Evening	Bike	20	2	84
4	741	9.52	Foggy	Low	Night	Scooter	28	1	59
5	661	7.44	Rainy	Medium	Afternoon	Scooter	5	1	37
6	412	19.03	Clear	Low	Morning	Bike	16	5	68
7	679	19.4	Clear	Low	Evening	Scooter	8	9	57
8	627	9.52	Clear	Low		Bike	12	1	49
9	514	17.39	Clear	Medium	Evening	Scooter	5	6	46
10	860	1.78	Snowy	Low	Evening	Car	20	6	35
11	137	10.62	Foggy	Low	Evening	Scooter	29	1	73
12	812	16.86	Snowy	Medium	Afternoon	Car	13	4	88
13	77	15.54	Clear	Low	Night	Bike	29	1	76

Preprocessing steps

- Check that all required feature and target columns exist.
- Convert numeric feature and target columns to numeric values; invalid entries become missing values.
- Remove rows with a missing target, because the model cannot learn from an unknown delivery time.
- Fill missing numeric input values using the median.
- Fill missing categorical values using the most frequent category.
- Convert categorical inputs into one-hot encoded columns.
- Keep preprocessing and model training together in a scikit-learn Pipeline so prediction uses the same transformations.
- Split the usable rows into 80% training and 20% testing with random_state=42.

5. Tools and Technologies used

FoodRescue360 is developed using modern web development technologies, database management tools, and machine learning techniques. These technologies work together to provide a user-friendly interface, store project data efficiently, and predict food delivery time.

5.1 Frontend Technologies

- **HTML:** Provides the basic structure of the web application.
- **CSS:** Used to design the user interface, including layouts, colors, spacing, and styling.
- **JavaScript:** Adds interactivity and dynamic functionality to the application.
- **React.js:** Used to build reusable UI components and manage the frontend interface.
- **Vite:** Provides a fast development environment for running the React application.

5.2 Backend Technologies

- **Python:** The primary programming language used for backend development and machine learning.
- **FastAPI:** Used to develop backend APIs that connect the frontend, database, and machine learning model.
- **Uvicorn:** Runs the FastAPI application as a local development server.

5.3 Database Technologies

- **MongoDB Atlas:** A cloud-based NoSQL database service used to store application data.
- **PyMongo:** A Python library that allows the backend to connect to MongoDB and perform database operations such as inserting, retrieving, updating, and deleting records.

5.4 Machine Learning Technologies

- **Pandas:** Used to load, organize, and manipulate the dataset.
- **NumPy:** Supports numerical operations and data processing.
- **Scikit-learn:** Used to preprocess the dataset, split it into training and testing sets, train the model, and evaluate its performance.

- **Random Forest Regressor:** The machine learning algorithm used to predict delivery time in minutes.
- **SimpleImputer:** Handles missing numerical and categorical values.
- **OneHotEncoder:** Converts categorical features into a numerical representation suitable for machine learning.

6. Machine-Learning Model Implementation

Why Random Forest Regressor?

The model predicts a number of minutes, so this is a regression problem. A Random Forest Regressor combines predictions from many decision trees. The approach can learn non-linear relationships between distance, traffic, weather, vehicle type, preparation time, and delivery duration without requiring the relationship to be a straight line.

Model configuration

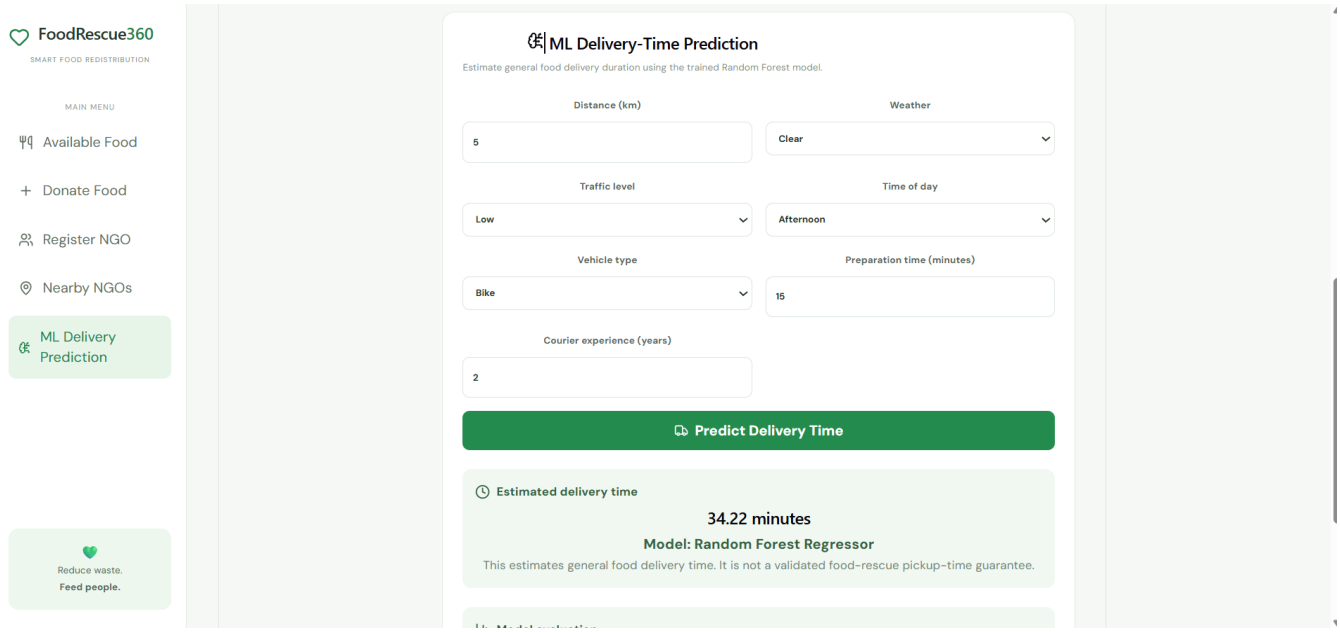
Setting	Value in code
Estimator	RandomForestRegressor
Number of trees	200
Random seed	42
Minimum samples per leaf	2
Parallel processing	n_jobs = -1
Target	Delivery_Time_min
Test size	20%

Training and prediction flow

- Load the CSV and select the input features and target.
- Create separate preprocessing pipelines for numeric and categorical columns.
- Fit the preprocessing/model pipeline on the training set only.
- Predict delivery times for the held-out test set.
- Calculate R^2 and MAE by comparing predictions with actual delivery times.

Example input

A sample request can use distance = 5 km, weather = Clear, traffic = Low, time of day = Afternoon, vehicle = Bike, preparation time = 15 minutes, and courier experience = 2 years. The previously recorded example prediction was 34.22 minutes. Your local output can vary if the data or model code changes.



7. Evaluation Results and Explanation of metrics

The FoodRescue360 project uses a Random Forest Regressor to predict food delivery time in minutes. To check the model's performance, the dataset was divided into two parts. Out of 1,000 records, 800 were used for training and 200 were used for testing.

The model was evaluated using R^2 Score and Mean Absolute Error (MAE). Accuracy, Precision, Recall, and F1-score were also calculated for a separate classification task using a 20-minute threshold.

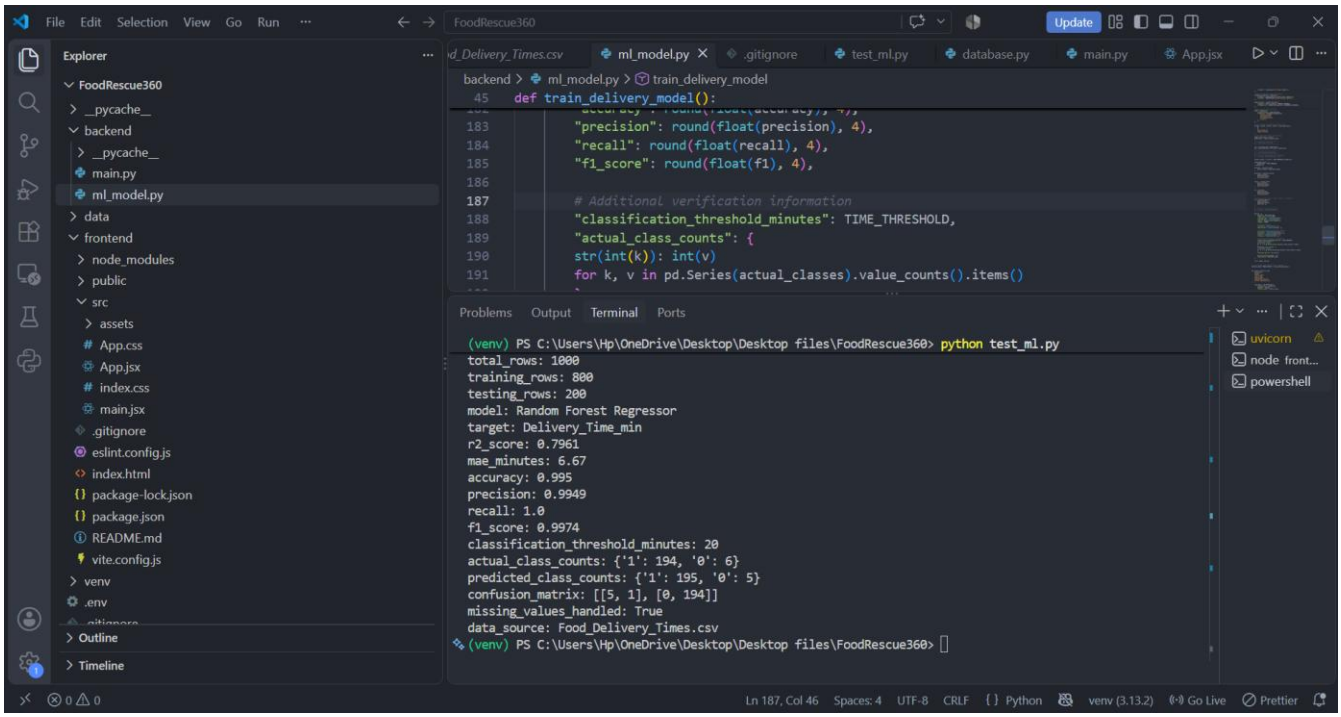


Figure 4. Model evaluation results displayed in the VS Code terminal.

7.2 Understanding the result

R^2 Score: The R^2 Score of 0.7961 indicates that the model explains approximately 79.61% of the variation in delivery times in the test dataset. It does not mean that 79.61% of the predictions are exactly correct.

Mean Absolute Error (MAE): The MAE of 6.67 minutes means that the predicted delivery times differed from the actual delivery times by approximately 6.67 minutes on average.

Accuracy: The accuracy of 99.50% means that 199 out of 200 test records were classified correctly according to the selected 20-minute threshold.

Precision: The precision of 99.49% indicates that almost all records predicted as delayed belonged to the delayed class.

Recall: The recall of 100% means that all records belonging to the delayed class in the test dataset were correctly identified.

F1-score: The F1-score of 99.74% combines precision and recall into a single measure and shows how well the threshold-based classification performed.

8. Limitations, Improvements and Conclusion

Current limitations

- The dataset describes general food delivery, not food-rescue pickup operations.
- The metrics come from one train/test split and are not a guarantee of performance on new data.
- Classification metrics do not apply to the current continuous regression target.
- Predictions depend on data quality and whether new inputs resemble the training data.
- The prototype does not verify food safety, guarantee collection, or confirm NGO capacity.
- The NGO verification route is demo-only and must be protected before production deployment.

Future improvements

- Collect real food-rescue data with actual pickup and collection timestamps.
- Add secure donor/NGO accounts and role-based access control.
- Add expiry notifications and food-safety checks.
- Improve NGO matching using verified coordinates and capacity.
- Compare regression models using the same validation approach and report additional diagnostics.
- Add automated tests, logging, monitoring, and a deployment guide.

Conclusion

FoodRescue360 brings together a community-focused use case and practical full-stack technologies. React provides the interface, FastAPI manages the application workflow, and MongoDB Atlas stores food listings, NGO profiles, and imported delivery records through PyMongo. The project demonstrates

database CRUD operations, geospatial indexing, request validation, and a Random Forest regression pipeline.

The recorded model results are $R^2 = 0.7961$ and MAE = 6.67 minutes. They provide a useful starting point for a classroom demonstration, but the training data is not specific to food rescue. The most important next step is to collect real rescue-pickup data and validate the model before using its predictions to guide real collection decisions.

References

- MongoDB Documentation — https://www.mongodb.com/docs/
- PyMongo Documentation — https://www.mongodb.com/docs/
- FastAPI Documentation — https://fastapi.tiangolo.com/
- React Documentation — https://react.dev/
- pandas Documentation — https://pandas.pydata.org/docs/
- scikit-learn Documentation — https://scikit-learn.org/stable/

Appendix - Website Screenshot Guide

Figure A1. Dashboard

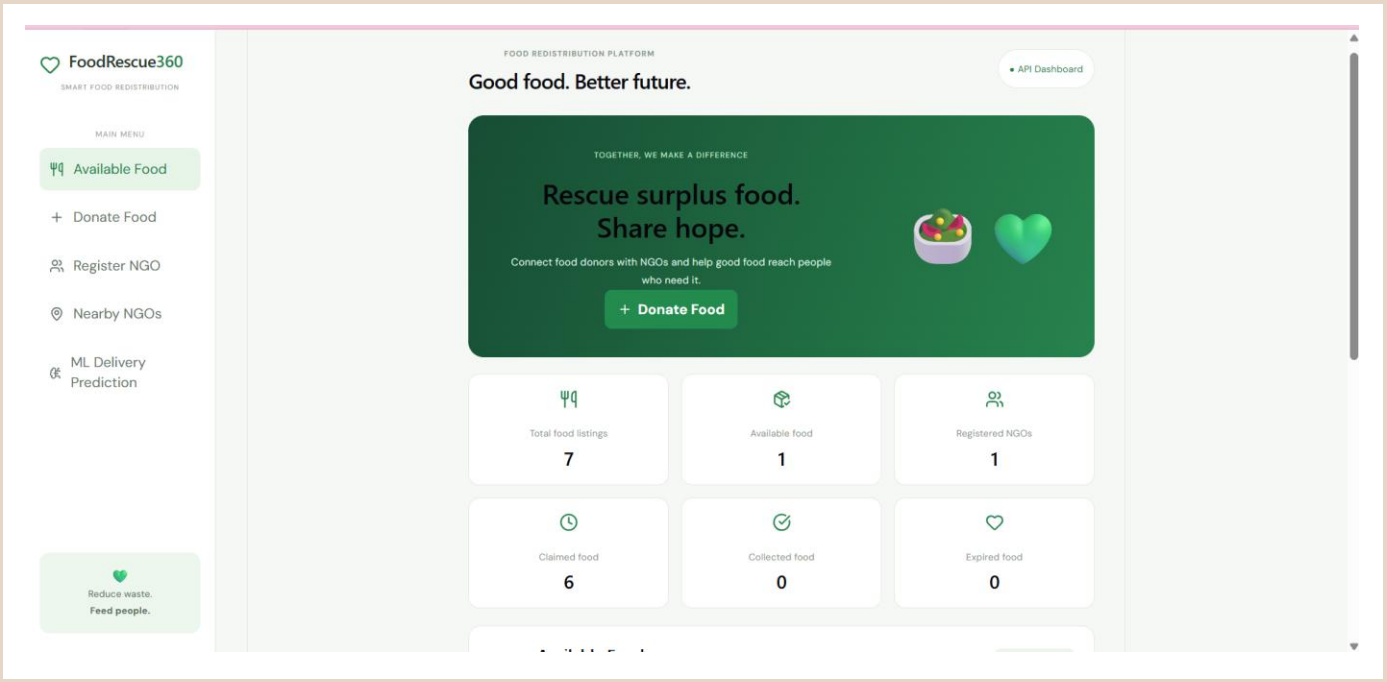


Figure A2. Add Food Listing

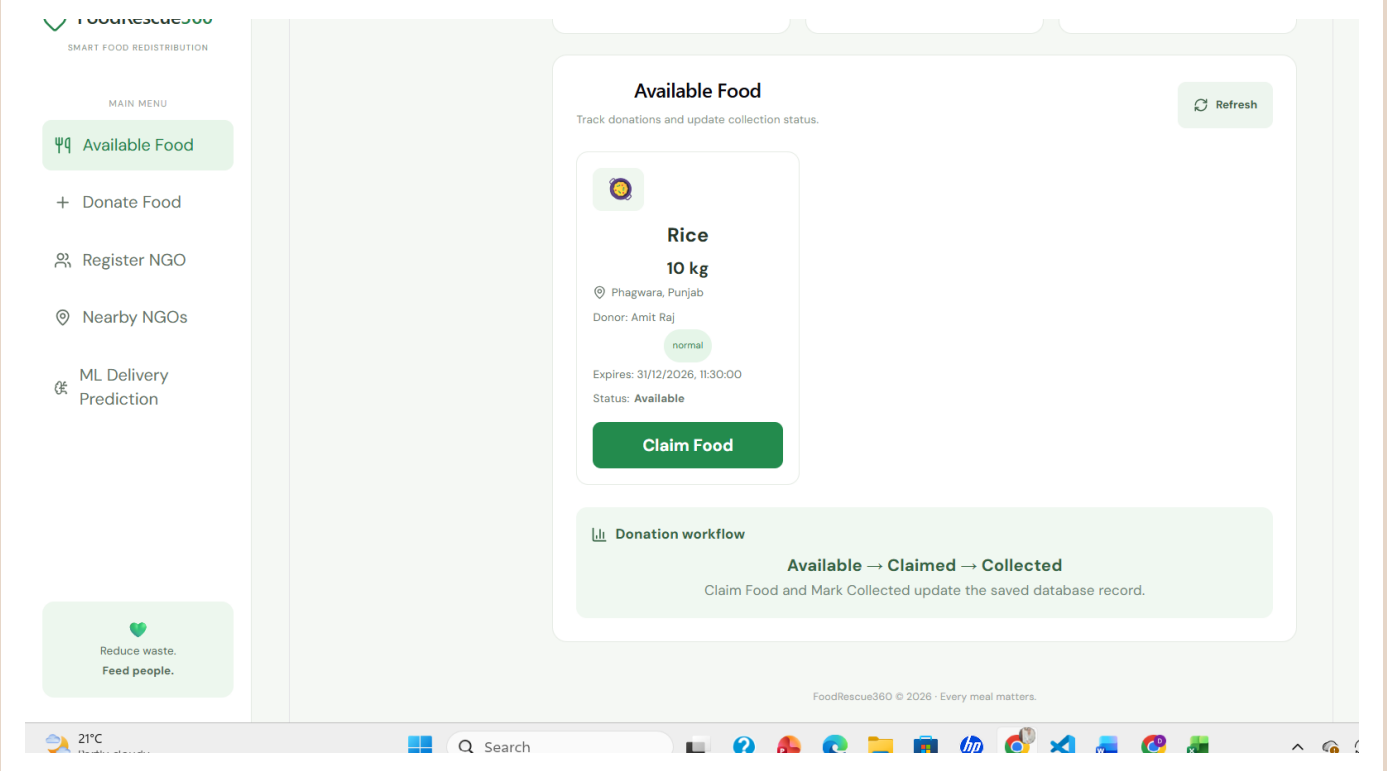


Figure A3. Donate Food

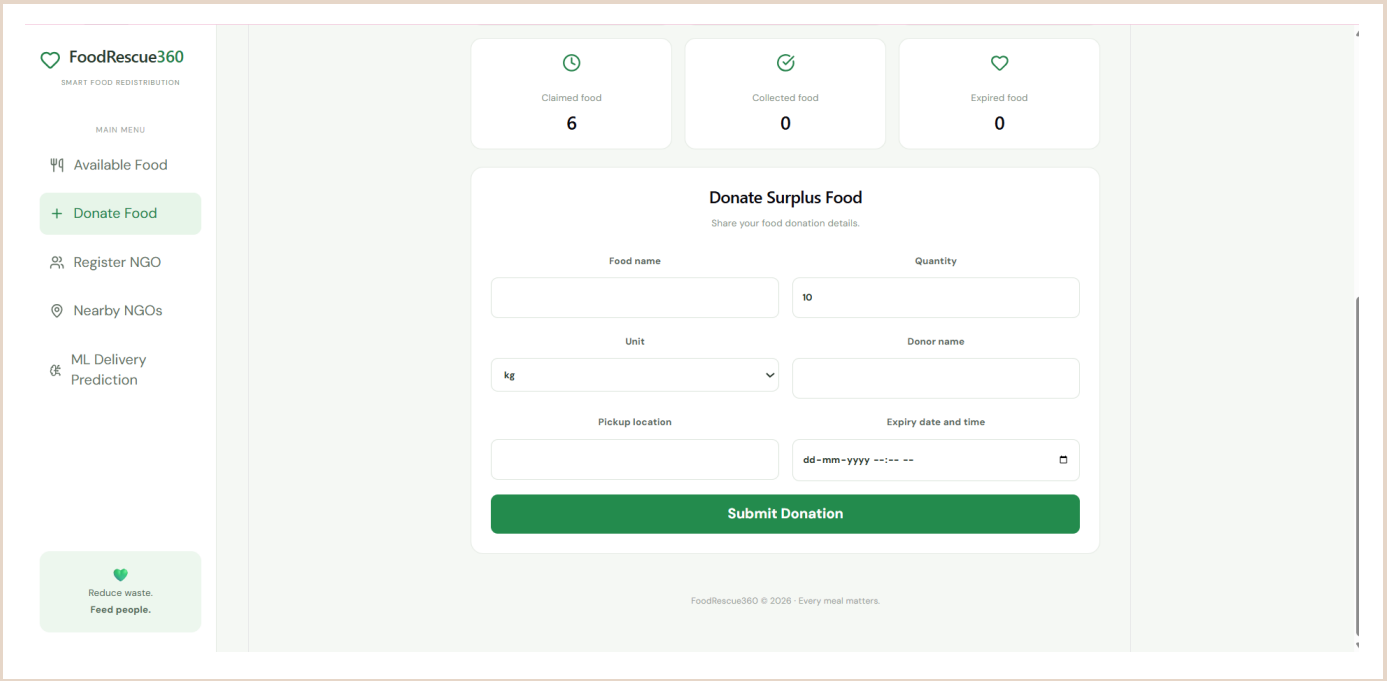


Figure A4. Nearby NGOs

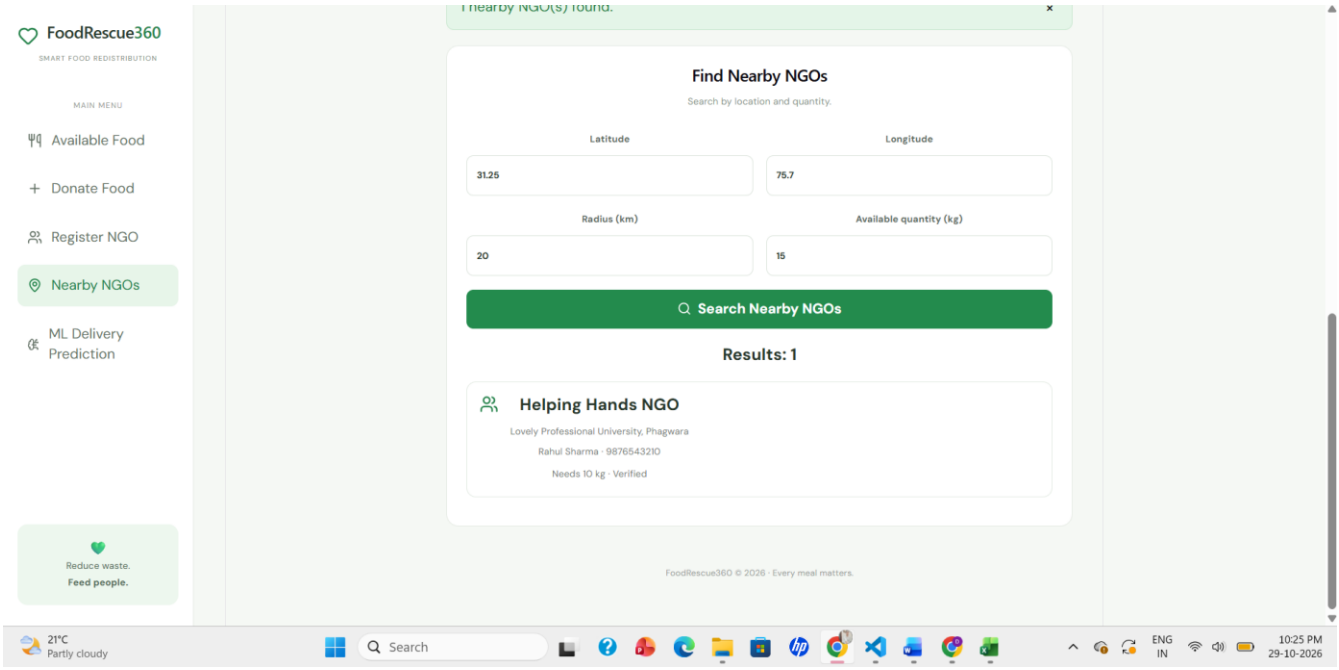


Figure A5. Delivery Prediction

